FluidLSM: A Fluid Log-Structured Merge Tree for Shifting Workloads

Shubham Kaushik Steven Yang Boao Chen Subhadeep Sarkar

LSM Background

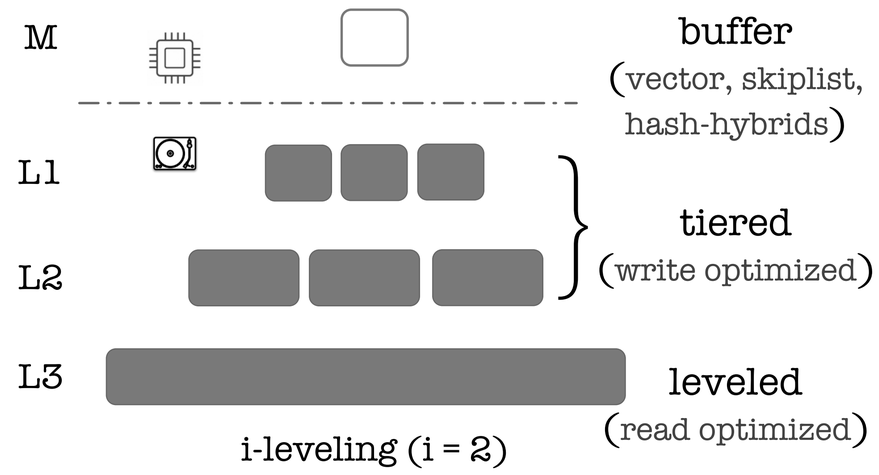
LSMs are Everywhere

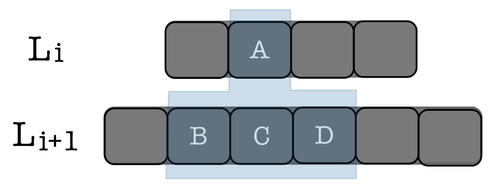
read-optimized | write-optimized

RocksDB, TiDB, levelDB, BigTable, FoundationDB, cassandra, speedb, SCYLLA, APACHE HBASE

What if the **workload properties** keep **changing** over time ?

i-Leveling Data Layout in LSM





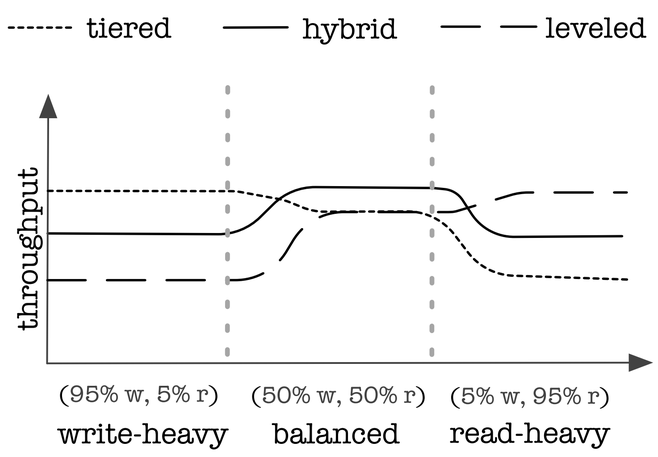
file picking policies
1) minimum overlap
2) round robin
3) first in, first out
4) oldest smallest seq first
...

commercial LSM engine exposes hundreds of tuning knobs!

each design choice and knob tuning impacts LSM performance

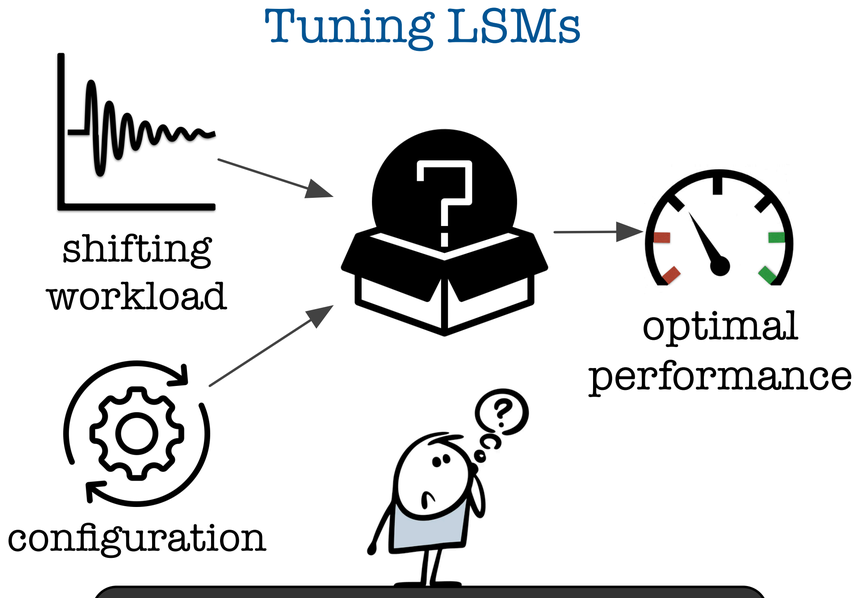
Problem

Motivation



performance varies widely with shifting workload and different data layouts

Tuning LSMs



What is the optimal setting under shifting workload?

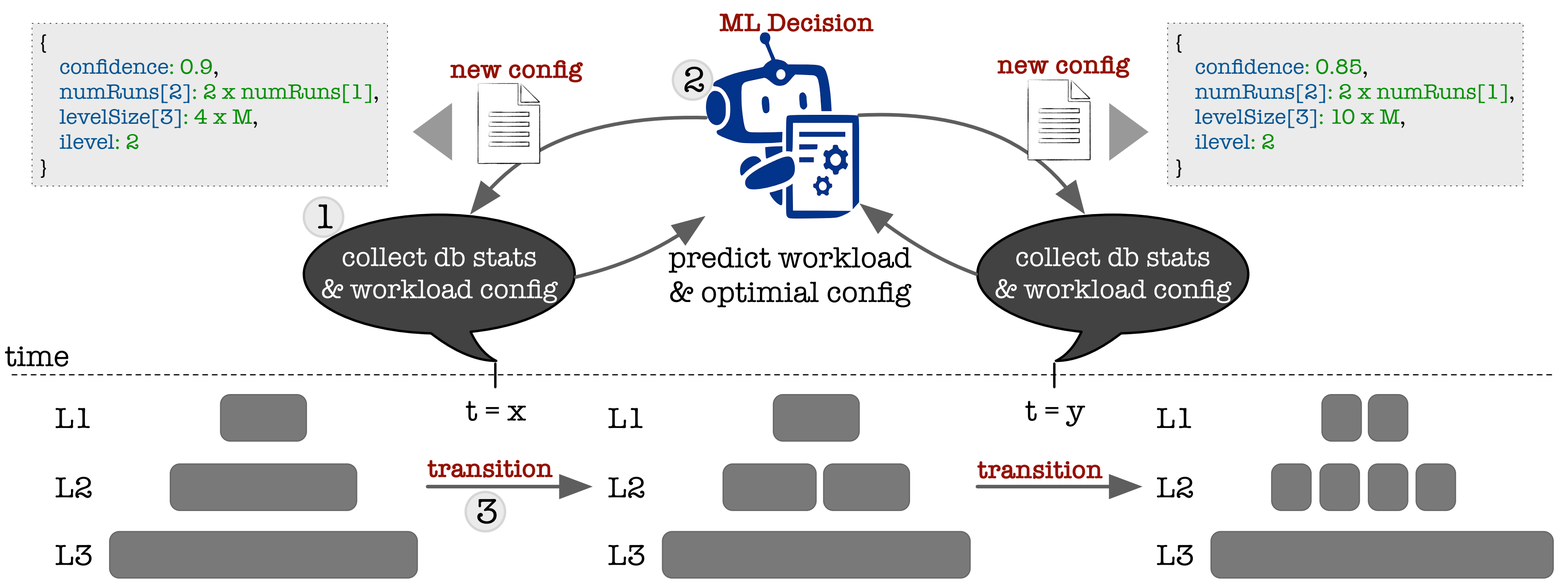
Design Challenges

- vast design space
- changing workloads
- cost model is not enough
- hand tuning not feasible
- high transition overhead

FluidDB

Overview

y - x = one epoch



offers continuous workload-aware adaptation

stable performance under workload shifts

online LSM shape transition; no-restart

Contributions

hybrid data layout
first i-level uses tiered layout

configurable tiers per level
variable tiers between levels

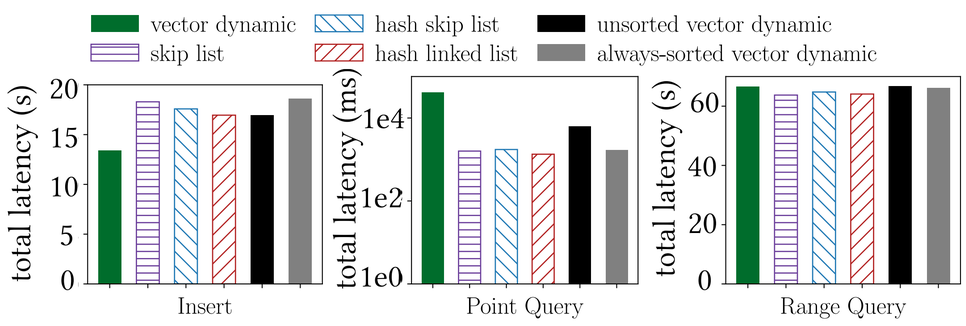
different size ratio per level
configurable sizes between levels

compaction granularity
configurable compaction size & file picking policy

Evaluation

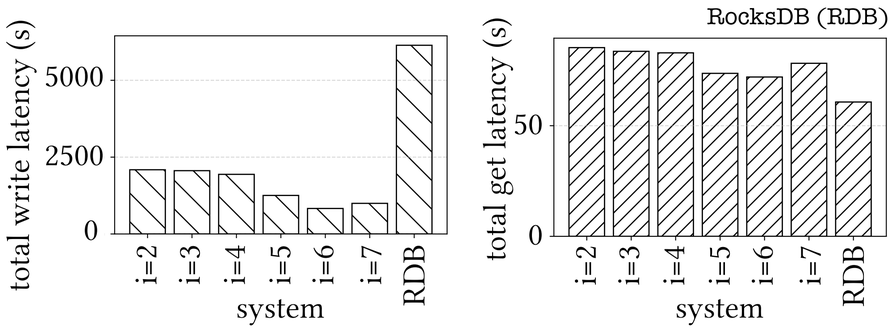
Preliminary Results

db size: 1GB; buffer: 4MB; entry size: 128B; size ratio: 2; workload: inserts 450K; point queries 10K; range queries: 10K with selectivity: 0.1



Choice of buffer affects LSM read and write performance

db size: 1GB; buffer: 64KB; entry size: 1KB; size ratio: 4; workload: 95% inserts 5% point queries



Increasing tiered levels lowers write latency, trading read performance

Future Work

- decision model
- transition b/w layouts
- predicting workload patterns

Brandeis UNIVERSITY
Michtom School of Computer Science